

Q-Rex: AI-Assisted Virtual Camera Tracking for FSOC Coarse Alignment

Technical Report — ISRO Problem Statement 26169 (Smart Automation / Space Technology)

1. Problem understanding

Free-space optical communication (FSOC) links between mobile platforms (satellites, UAVs) use laser beams that are only micro-radians wide. Establishing a link is a pointing-acquisition-tracking (PAT) problem with two stages: *coarse alignment* (put the remote terminal's beacon inside the camera field of view and keep it there) and *fine alignment* (a fast steering mirror closes the last arc-seconds). This project implements the coarse stage entirely in software, so that algorithms can be developed without a camera, gimbal or optics.

The software must: generate a configurable virtual scene; generate one or more moving beacons; implement a movable virtual camera with realistic pan/tilt limits; detect the beacon automatically; track it with computer vision; control the camera; inject atmospheric turbulence, platform vibration, camera jitter and sensor noise into the feed; and display performance in real time. Deliverables are an application, source code, this report, a user manual, and an automatically generated performance log.

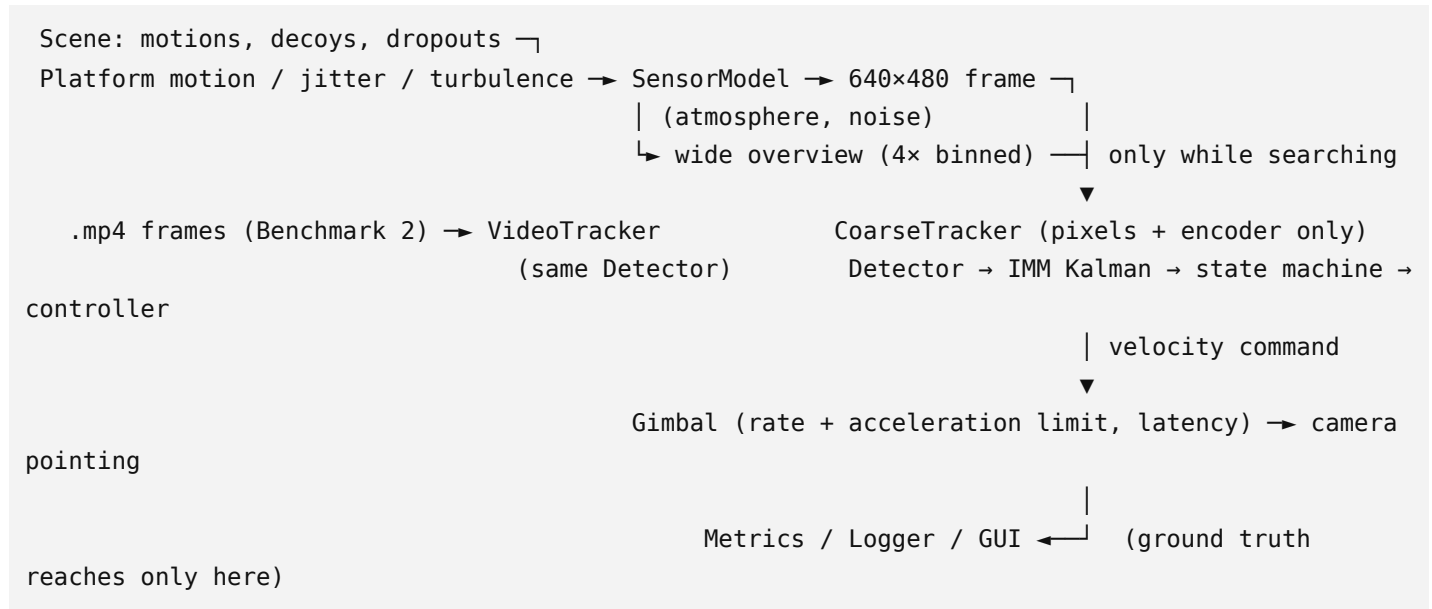
1.1 Numbers that shape the design

Quantity	Value	Consequence
Camera	640×480, FOV 4°×3°	160 px/° (IFOV 0.00625°)
Screen	2000×2000 px	12.5° across; the camera sees only 7.7 % of it
Slew limit	5°/s (default)	800 px/s = 26.7 px/frame at 30 Hz
Acquisition	≤ 2 s	A raster scan needs ≥ 12 s, so scanning alone cannot meet the spec (§6.1)
Platform motion	±20 px/frame	600 px/s: 75 % of the slew budget before the target even moves
Jitter	±20 px/frame	Every single-frame measurement is corrupted by up to 20 px

2. Requirements and compliance

Every row of the specification table is implemented, configurable (GUI and `config.js`) and covered by an automated test (`tests/spec.test.mjs`, 26 tests); the full mapping is in `TRACEABILITY.md`. The optional colour camera is implemented as three independently-noisy RGB channels (reddish beacon on a bluish-grey background); the tracker consumes the luminance image.

3. System architecture



Principles: (1) the tracker never sees ground truth — a unit test asserts that its only inputs are the frame, the encoder position and the clock; (2) simulation time is frame-locked ($dt = 1/\text{fps}$) and independent of wall time; timing metrics use simulation time, performance metrics use wall time; (3) every random subsystem has its own seeded stream, so the same seed reproduces the same log; (4) the engine has no DOM dependency, so the identical code runs in the browser, under Node (benchmarks, tests) and in the desktop shell.

4. Software modules

Module	Responsibility
<code>rng.js</code>	Seeded streams (mulberry32 + splitmix), Gaussian, Poisson
<code>config.js</code>	All parameters with spec defaults/ranges, atmosphere presets, scenario list
<code>motion.js</code>	Straight, circular, figure-8, random, spiral, sinusoidal, user waypoints (Catmull-Rom smoothed); platform motion models. Curves are re-parameterised by arc length so the configured speed is exact

Module	Responsibility
render.js	Sensor model: analytic anti-aliased beacon (erf/box integral, sub-pixel exact), PSF blur, atmosphere, rain streaks, Poisson/Gaussian/salt-and-pepper noise, 4x binned overview
detector.js	Impulse rejection, multi-scale matched filter with CFAR, validation, centroiding
kalman.js	Two-model IMM Kalman filter with adaptive noise
tracker.js	State machine, acquisition, gating, controller
gimbal.js	Pan/tilt dynamics
simulation.js	Closed loop, ground-truth recording
metrics.js	Metric definitions, JSON/CSV/HTML reports
videoTracker.js, videoRunner.js	Benchmark-2 path
App.jsx, fields.js	GUI

5. Virtual environment and disturbance models

- **Beacon:** square (default), circle or diamond, 5–20 px, peak intensity 230 on a background of 20 grey levels. Rendered analytically: the exact integral of a Gaussian-blurred box over each pixel, so sub-pixel motion is faithful and there is no aliasing.
- **Noise** (selectable, combinable): Poisson shot noise, then Gaussian ($\sigma \leq 20$ grey levels), then salt-and-pepper ($\leq 10\%$ of pixels, half 0, half 255), then clipping.
- **Jitter:** per-frame i.i.d. shift of the line of sight, truncated Gaussian, hard bound $\pm J$ ($J \leq 20$ px).
- **Platform motion:** line-of-sight offset whose per-frame displacement never exceeds $P \leq 20$ px: linear (default, reflected inside ± 400 px with smooth, jerk-limited turn-arounds), circular, random, spiral, figure-8. The platform offset is *not* given to the tracker.
- **Atmosphere:** $I' = C \cdot I + B$ plus PSF blur: haze ($C\ 0.7$, $B\ +18$, $\sigma\ 1.0$), fog (0.45 , $+35$, 2.0), rain (0.75 , $+5$, 0.8 , plus streaks), low light (0.35 , -5 , shot-noise $\times 3$), scaled by a severity slider.
- **Turbulence:** beam wander (Ornstein–Uhlenbeck), log-normal scintillation, extra PSF blur.
- **Beacon dropouts:** the beacon disappears periodically (cloud/occlusion) to exercise re-acquisition.

6. Tracking methods

6.1 Acquisition: two-tier sensing

The narrow camera covers 7.7 % of the screen, so a boustrophedon scan needs ≥ 5 swaths $\times 2000 \text{ px} \div 800 \text{ px/s} \approx 12.5 \text{ s}$. The $\leq 2 \text{ s}$ requirement is therefore unattainable by scanning with the narrow camera alone. Q-Rex uses a wide-area *overview* view (the screen binned $4\times$, 500×500) **only while searching or re-acquiring**, exactly like the Benchmark-2 videos which cover the complete screen; tracking uses only the 640×480 camera. A raster-scan mode without the overview is also implemented (Acquisition = Raster scan) and is reported separately.

6.2 Detection (computer vision)

1. **Impulse noise:** if more than 0.3 % of sampled pixels are saturated, a 3×3 median filter (border-safe) is applied; a second pass for dense impulse noise ($> 2 \%$).
2. **Multi-scale matched filter:** box filters at five scales spanning 5–20 px are evaluated from an integral image. A box of the right size is the matched filter for a flat-topped blob in white noise.
3. **Self-calibrated CFAR:** per scale, background and noise are the median and MAD of the *filter output* itself, so the threshold adapts to correlation, clipping and noise type with no knowledge of the noise model. Default threshold 6σ (raised by $2\sigma/6\sigma$ under impulse noise, whose residual clusters have heavier tails).
4. **Validation:** thresholded blob area must fill $\geq 35 \%$ of the matched box (rejects surviving impulse pixels); eccentricity from second moments rejects rain streaks when SNR allows.
5. **Centroid:** iterative weighted centroid (threshold at $\max(0.3\cdot\text{amplitude}, 2\sigma)$, ring-median local background, four re-centring iterations). Measured centroiding RMSE is 0.02–0.2 px.
6. **ROI processing:** in tracking mode only a window around the predicted position is processed (64–160 px half-width), which keeps tracker cost at 2–8 ms/frame.

6.3 Estimation: IMM Kalman filter

Two models (constant velocity and constant acceleration) are mixed by an interacting-multiple-model filter (transition probability 0.04). Measurements are world positions (encoder position + centroid offset), so the filter sees target-minus-platform motion directly. Features: chi-square gating, Huber inflation for large innovations, **measurement noise observed from second differences** of consecutive measurements ($\text{var}(z_k - 2z_{k-1} + z_{k-2}) = 6R$ for smooth motion, using a median-absolute-deviation estimate, so jitter level is learned online and independently of the motion model), and **process noise adapted from normalised innovations**

(raised immediately on a manoeuvre, decayed during steady motion, never below a safe floor).

6.4 State machine

SEARCH (overview detection, two consecutive consistent hits) → SLEW (camera slews to the estimate; the overview keeps steering; one narrow-camera detection agreeing with the prediction confirms) → TRACK → COAST (prediction only, with overview assistance) → REACQUIRE (search with a prior favouring the last known position, weight $e^{(-d/350 \text{ px})}$). Timeouts and an "arrived-but-unseen" exit prevent deadlock. Decoys are rejected by gating on the prediction; the designated beacon is the strongest candidate at first lock.

6.5 Control

Per axis: $u = g \cdot (\hat{x}_{\text{aim}} - p)/dt + (1 - g) \cdot \hat{v}$ with $\hat{x}_{\text{aim}}$ the latency-compensated prediction ($g = 0.85$), i.e. feed-forward from the estimated velocity plus shaped feedback. A minimum-time stopping profile limits the correction to $\sqrt{(2 \cdot a \cdot |d|)}$ so the camera brakes before the target instead of overshooting. The gimbal model enforces the rate limit (5–10°/s), an acceleration limit (default 20°/s², the statement leaves it open) and optional command latency. A pointing range of ± 450 px beyond the screen lets platform offsets be compensated at the screen edge (found by testing: without it, the mount saturated at the boundary).

7. AI methods — and an honest statement of scope

The "AI" in Q-Rex is **model-based adaptive estimation and robust statistical detection**, not a trained neural network: IMM multiple-model inference, online noise identification (MAD of second differences, MAD-calibrated CFAR) and innovation-driven adaptation of the process noise. These are learned-from-data-online components, selected because they run in 2–8 ms per frame, need no training set, behave predictably under all required disturbances, and let the centroiding error stay at the 0.1 px level. A learned heat-map detector (small CNN trained on simulator data) is planned as future work (§10) but is **not** part of the delivered system; we make no claim that it exists.

8. Metric definitions

- **Tracking (boresight) error:** distance of the true beacon from the image centre, excluding the random per-frame jitter the camera itself suffers (also reported

including jitter); steady-state window = from the first frame within 10 px after lock; evaluated on frames where the beacon is inside the FOV.

- **Centroiding error:** detector centroid vs true beacon position in the same frame (mean, RMSE, P95, max).
- **Acquisition time:** simulation time from start to the first confirmed lock inside the narrow FOV (also `timeToCentre`).
- **Target loss:** share of frames since first lock in which the beacon is outside the FOV. **Lock retention:** share of frames in TRACK/COAST, beacon in FOV and estimate within 60 px.
- **Re-acquisition time:** from leaving TRACK to the next TRACK (maximum over events).
- **FPS / processing time:** wall-clock of tracker + controller per frame; scene rendering stands in for the sensor and is excluded (end-to-end FPS is logged too).

The automatically generated report (JSON, HTML and per-frame CSV) contains all of these plus simulation duration and gimbal saturation percentage.

9. Test methodology and results

Automated tests (`npm test`, 38 tests): determinism (same seed $\Rightarrow$ identical log), motion speed exactness and screen containment, platform step bound, detector accuracy under all noise types and sizes, false-alarm rate, closed-loop pass/fail, scan acquisition, decoy rejection, video path, and a one-test-per-row compliance suite for the specification table (`tests/spec.test.mjs`), including forced re-acquisition tests (camera knocked 450 px off; 0.5 s beacon dropouts).

Scenario benchmark (`npm run bench -- 3 20`, 3 seeds $\times$ 20 s, default gimbal):

Scenario	Acq. s	Err px	Cent. RMSE px	Loss %	Lock %	Re-acq s	ms/frame	Pass
A1 circular	0.62	0.4	0.05	0	100	0	3.0	3/3
A2 straight line 300 px/s	0.68	0.1	0.03	0	100	0	6.3	3/3
A3 figure-8	0.66	0.4	0.06	0	100	0	3.0	3/3
A4 random	0.59	0.4	0.06	0	100	0.03	2.9	3/3
B1 Gaussian $\sigma=20$	0.61	1.0	0.11	0.1	99.9	0	8.0	3/3
B2 salt & pepper 10 %	0.62	0.4	0.11	0	100	0	5.2	3/3
B3 Poisson	0.61	0.8	0.09	0.1	99.9	0	6.2	3/3
B4 jitter ± 20 px/frame	0.62	6.8	0.05	0	100	0	2.5	3/3

Scenario	Acq. s	Err px	Cent. RMSE px	Loss %	Lock %	Re-acq s	ms/frame	Pass
B5 platform 20 px/frame	1.26 (max 1.93)	0.8	0.05	0	98.3	0.53	2.0	3/3
B6 fog	0.61	0.4	0.06	0.1	99.9	0	3.1	3/3
B7 rain	0.62	0.4	0.02	0	100	0	3.0	3/3
B8 low light	0.62	0.5	0.05	0	100	0	2.9	3/3
B9 decoys (3 targets)	0.62	0.4	0.06	0	100	0	2.9	3/3
B10 beacon dropouts 0.5 s	0.62	0.9	0.05	0	100	0.50	3.9	3/3
C1 all disturbances at maximum	1.24	19.2	1.14	0	98.5	0.03	16.1	0/3

Benchmark 2 path (browser end-to-end, `npm run e2e`): a noisy 2000×2000 @30 fps H.264 video (CRF 20, Gaussian $\sigma \approx 12$, 10 px beacon on a sinusoidal path, exact ground truth) is decoded frame-by-frame by the GUI, tracked, and compared with the ground-truth CSV: acquisition 0.033 s, lock retention 100 %, centroiding RMSE 0.072 px (P95 0.16, max 0.21), 5.2 ms/frame. Organiser videos were not available and have not been tested.

Why C1 cannot reach 10 px (analysis, not just observation). With ± 20 px/frame jitter modelled as truncated Gaussian noise ($\sigma \approx 8$ px per axis) on every measurement, a *causal* estimator of a smooth trajectory has an end-point error of roughly $\sigma \cdot \sqrt{9/N}$ for a quadratic fit over N frames (≈ 4.4 px per axis, ≈ 5.5 px mean magnitude at $N = 30$); a shorter window lowers lag but raises noise. Jitter alone therefore leaves ≈ 7 px mean error (measured 6.7–7.1 px, independent of every filter setting we swept: process-noise floor, model set including a constant-jerk model, control gain). Adding a 20 px/frame platform drift forces a wider filter bandwidth, which raises the error to ≈ 15 px. This floor is a property of the disturbance, not a tuning gap; any single-frame estimator without a static scene reference faces it.

Honest limitations. C1 (every disturbance at its maximum simultaneously) fails the 10 px tracking-error target: with ± 20 px/frame jitter on every measurement and a 20 px/frame platform drift, the estimator cannot be both smooth enough to suppress jitter and fast enough to follow the platform; the beacon is never lost (0 % loss) but is centred only to ~ 19 px mean. Acquisition with the maximum platform offset is within 0.07 s of the 2 s limit. The scan-only acquisition mode cannot meet 2 s (physical limit). Acceleration limits are our choice; with a weaker mount the maximum-platform cases saturate (the log reports saturation percentage). All results are from our own simulator.

10. Future improvements

- Trained heat-map detector with sub-pixel offset and predicted variance, fused by inverse-variance weighting, with the classical detector as fallback; synthetic-data training with domain randomisation.
- Coordinated-turn model in the IMM, and a smoother (RTS) for offline Benchmark-2 analysis (reported separately, non-causal).
- Track-before-detect for very low SNR; background-texture ego-motion estimation to cancel jitter when the scene has features.
- MPC with rate and acceleration constraints; saturation-aware priority logic (keep in FOV first, centre second).
- Optional colour camera; Monte-Carlo sweeps with confidence intervals; native installers for Windows/macOS.

Appendix A — Parameter reference (defaults and ranges)

Parameter	Default	Range	Spec row
screenSize	2000 px	2000–6000	1
cameraType	mono	mono, colour	2
camW × camH	640 × 480	160–1280 × 120–960	3
fovX (vertical = fovX·camH/camW)	4° (→ 4°×3°)	1–12°	4
fps	30 Hz	30–120	5, 15
maxPanSpeed, maxTiltSpeed	5 °/s	5–10	13, 14
maxAccel	20 °/s ²	5–100	not specified
numTargets	1	1–6 (extra = decoys)	8
targetShape	square	square, circle, diamond	9
targetSize	10 px	5–20	10
startMode	random	random, user (startX, startY)	11
motion	circular	linear, circular, figure8, random, spiral, sinusoidal, waypoints	12
targetSpeed	200 px/s	20–700	design choice (≤ 60 % of slew)
saltPepperDensity	0.10	0–0.10	21.1
gaussianSigma	10 grey levels	0–20	21.2
poisson	off	on/off	21.1
jitterMax	10 px/frame	0–20	21.3

Parameter	Default	Range	Spec row
atmosphere, atmosphereLevel	clear, 1.0	clear, haze, fog, rain, lowlight; 0-1	21.4
platformModel, platformSpeed	linear, 10 px/frame	linear, circular, random, spiral, figure8; 0-20	21.5
turbulence	0	0-1	—
dropout, dropoutEvery, dropoutLen	off, 6 s, 0.5 s	—	19 (testing aid)
acquisition	overview	overview, scan	16
controlGain	0.85	0.2-1	—
detectThreshold	6 σ	4-10	—
coastFrames	20	3-90	19

Grey-level interpretation: the statement gives the maximum noise standard deviation as "20 pixels"; we interpret it as 20 grey levels on a 0-255 scale (a question for the mentors; the setting is a single parameter if a different reading is intended).

Appendix B — Test catalogue

Engine tests (`tests/engine.test.mjs`, 12): RNG determinism; configuration clamping to spec ranges; all seven motions stay on screen at the configured speed (± 6 %); platform displacement ≤ 20 px/frame for all five models; sub-pixel centroid under Gaussian $\sigma 20$, salt & pepper, fog (sizes 5, 10, 20); false-alarm rate on empty noisy frames; closed-loop nominal pass; identical logs for identical seeds; scan acquisition; decoy rejection; video tracker on a synthetic 2000×2000 stream; defaults equal the specification.

Specification tests (`tests/spec.test.mjs`, 26): one or more tests for each table row (screen size, monochrome frame, resolution, FOV, update rate, initial position, target count, shapes, sizes, initial location, all motions, speed limits at 5 and 10 °/s, update interval, acquisition/tracking error/target loss/re-acquisition/FPS on the four mandatory motions, re-acquisition after the camera is knocked off and after beacon dropouts, every noise type and combination, noise σ and jitter really applied and bounded, atmosphere contrast reduction, all five platform models, turbulence, decoys, performance-log fields, and the "ground truth never reaches the tracker" interface test).

End-to-end test (`scripts/e2e-video.cjs`): synthesises a 2000×2000 H.264 video with exact ground truth, drives the real GUI in headless Chromium, uploads video and ground-truth CSV, and asserts RMSE < 1 px, lock ≥ 99 % and acquisition ≤ 0.2 s (measured 0.072 px, 100 %, 0.033 s).

Benchmark (*scripts/bench.mjs*): the 15-scenario suite of §9.

Appendix C — Reproducing every number in this report

```
npm install
npm test           # 38 tests
npm run bench -- 3 20 # section 9 table (≈ 15 min); seeds 1..3
npm run e2e        # Benchmark-2 browser test
npm run build:single # dist-single/q-rex.html
npm run build:exe   # release/q-rex-<platform>/
```

All runs are seeded; identical seeds give bit-identical logs.